

Name:- Vaishnavi rajesh bomnale

PRN:-202501050063

Batch :- CH3

Sub :- EDS

Screenshot of practical no 4:

Practical lab assignment:

The image displays three screenshots of the CODETANTRA IDE interface, showing practical assignments for Pandas series, DataFrame, and file reading.

Screenshot 1: 4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

Code:

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5 # Create a Pandas series from the list of numbers
6 df = pd.Series(numbers)
7 # Grouping by even and odd numbers and calculating the mean
8 grouped = df.groupby(df%2==0).mean()
9
10 # Display the mean of even and odd numbers with labels
```

Test Results:

- 3 out of 3 shown test case(s) passed
- 3 out of 3 hidden test case(s) passed

Test case 1:

Expected output	Actual output
1 2 3 4 5 6 7 8 9 10	1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:	Mean of even and odd numbers:
Odd : 5.0	Odd : 5.0
Even : 6.0	Even : 6.0
dtype: float64	dtype: float64

Screenshot 2: 4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Change the column name to lowercase.

Code:

```
54
55 # Provided dictionary of lists
56 data = {
57     'Name': ['Alice', 'Bob', 'Charlie'],
58     'Age': [25, 30, 35],
59 }
60
61 # Convert the dictionary to a DataFrame
62 df = pd.DataFrame(data)
63
64 # Display the original DataFrame
```

Test Results:

- 1 out of 1 shown test case(s) passed
- 1 out of 1 hidden test case(s) passed

Test case 1:

Expected output	Actual output
Original DataFrame:	Original DataFrame:
Name Age	Name Age
0 Alice 25	0 Alice 25
1 Bob 30	1 Bob 30
2 Charlie 35	2 Charlie 35
New name: Susan	New name: Susan

Screenshot 3: 4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

Sample Test Cases

Code:

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep='\s+', headers=None, names=["Name", "Age", "Grade"])
6
7
8 # write your code here..
9 print("First five rows:")
10 print(data.head())
```

Test Results:

- 1 out of 1 shown test case(s) passed
- 1 out of 1 hidden test case(s) passed

Test case 1:

Expected output	Actual output
studentdata.txt	studentdata.txt
First five rows:	First five rows:
Name Age Grade	Name Age Grade
0 John 25 A	0 John 25 A
1 AIn 22 B	1 AIn 22 B
2 Emma 24 A	2 Emma 24 A

CODETANTRAHomeLearn Anywhere202501050063@mitaoc.ac.inSupportLogout

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	8	10	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	20	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	10	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 df['Date'] = pd.to_datetime(df['Date'])
10 df['Month'] = df['Date'].dt.strftime('%Y-%m')
11 df['Total Sales'] = df['Quantity'] * df['Price']
```

Average time: 0.032 s, Maximum time: 0.065 s
1 out of 1 shown test case(s) passed, 2 out of 2 hidden test case(s) passed

Test case 1: Expected output: sales_data.csv, Actual output: sales_data.csv
Best month: 2025-01, Total sales: \$1210.00

CODETANTRAHomeLearn Anywhere202501050063@mitaoc.ac.inSupportLogout

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	20	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	20	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	20	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 product_sales = df.groupby('Product')['Quantity'].sum()
10 # Find the product with the highest total quantity sold
11 best_product = product_sales.idxmax()
12 highest_quantity = product_sales.max()
```

Average time: 0.024 s, Maximum time: 0.054 s
1 out of 1 shown test case(s) passed, 2 out of 2 hidden test case(s) passed

Test case 1: Expected output: sales_data.csv, Actual output: sales_data.csv
Best-selling product: Product A, Total quantity sold: 23

CODETANTRAHomeLearn Anywhere202501050063@mitaoc.ac.inSupportLogout

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	8	10	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	20	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # write the code..
10 city_sales = df.groupby('City')['Quantity'].sum()
11 best_city = city_sales.idxmax()
```

Average time: 0.024 s, Maximum time: 0.049 s
1 out of 1 shown test case(s) passed, 2 out of 2 hidden test case(s) passed

Test case 1: Expected output: sales_data.csv, Actual output: sales_data.csv
City sold the most products: Los Angeles

CODETANTRAHomeLearn Anywhere202501050063@mitaoc.ac.inSupportLogout

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pairs that were sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	8	10	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	20	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanations:
Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

frequentl...

```
10
11 # write the code
12 date_products = {}
13
14 for date, group in df.groupby('Date'):
15     products = group['Product'].unique()
16     if len(products) > 1:
17         date_products[date] = products
18
19 # Count product pairs
20 pair_counter = Counter()
```

Average time: 0.026 s, Maximum time: 0.053 s
1 out of 1 shown test case(s) passed, 2 out of 2 hidden test case(s) passed

Test case 1: Expected output: sales_data.csv, Actual output: sales_data.csv
Product A and Product B: 2 times, Product A and Product C: 2 times, Product A and Product C: 2 times

CODETANTRAHomeLearn Anywhere202501050063@mitaoc.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

- Display the first 5 rows of the dataset.
- Display the last 5 rows of the dataset.
- Get the shape of the dataset (number of rows and columns).
- Get a summary of the dataset (using .info()).
- Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
- Check for missing values and display the count of missing values for each column.
- Fill missing values in the 'Age' column with the median age.
- Fill missing values in the 'Embarked' column with the most frequent value (mode()).
- Drop the 'Cabin' column due to many missing values.
- Create a new column, 'FamilySize', by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

Pas- senger id	Surv ived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
----------------------	--------------	--------	------	-----	-----	-------	-------	--------	------	-------	----------

Raw data Data:

titanicDat...

```
37 # Display the first 5 rows of the dataset
38 print(data.head())
39
40 # 2. Display the last 5 rows of the dataset
41 print(data.tail())
42
43 # 3. Get the shape of the dataset (number of rows and columns)
44 print(data.shape)
45
46 # 4. Get a summary of the dataset
47 print(data.info())
```

Average time: 0.215 s, Maximum time: 0.215 s
1 out of 1 shown test case(s) passed

Test case 1: Expected output: PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked, Actual output: PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked

CODETANTRAHomeLearn Anywhere202501050063@mitaoe.ac.inSupportLogout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 30/322

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcla ss	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bark ed

Sample Data:

Explorer

titanicDat...

```
10 print(data.groupby('Pclass')['Survived'].mean())
11
12 #2. Calculate the survival rate by embarked location (Embarked_S)
13
14 print(data.groupby('Embarked_S')['Survived'].mean())
15
16 #3. Calculate the survival rate by family size
17
18 print(data.groupby('FamilySize')['Survived'].mean())
19
20 #4. Calculate the survival rate by being alone
```

Average time0.109 s109.00 ms

Maximum time0.109 s109.00 ms

1 out of 1 shown test case(s) passed

Test case 10/9 msDebug

Expected output

Actual output

Pclass:1...0.629630Pclass:1...0.629630

2...0.4728262...0.472826

3...0.2423633...0.242363

Name: Survived, dtype: float64Name: Survived, dtype: float64

Embarked_S:Embarked_S:

Terminal

Test cases

CODETANTRAHomeLearn Anywhere202501050063@mitaoe.ac.inSupportLogout

4.2.8. Titanic Dataset Analysis and Data Cleaning - 40/351

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcla ss	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bark ed

Sample Data:

Explorer

titanicDat...

```
25 # 5. Percentage of children (Age < 18) who survived
26 children = data[data['Age'] < 18]
27 children_survival_rate = children['Survived'].mean()
28 print(children_survival_rate)
29
30 # 6. Percentage of adults (Age >= 18) who survived
31 adults = data[data['Age'] >= 18]
32 adults_survival_rate = adults['Survived'].mean()
33 print(adults_survival_rate)
34
35 # 7. Median age of survivors
```

Average time0.081 s81.00 ms

Maximum time0.081 s81.00 ms

1 out of 1 shown test case(s) passed

Test case 161 msDebug

Expected output

Actual output

female:....233female:....233

male:....109male:....109

Name: Sex, dtype: int64Name: Sex, dtype: int64

male:....468male:....468

female:....81female:....81

Name: Sex, dtype: int64Name: Sex, dtype: int64

Terminal

Test cases